

HRCore React Geliştirme Süreci ve Teknik Raporu

1. Projenin Genel Tanımı

HRCore, insan kaynakları süreçlerini merkezi bir web paneli üzerinden yönetmek için geliştirilmiş React tabanlı bir uygulamadır. Uygulama çalışan kayıtları, izin talepleri, maaş hesaplama, cihaz zimmetleri, doküman yönetimi ve rol bazlı erişim kontrolü gibi modülleri kapsar.

Projenin frontend tarafı React, TypeScript, Vite ve React Router ile geliştirilmiştir. Backend ve veri katmanı için Supabase kullanılmıştır. Supabase tarafında Authentication, PostgreSQL tabloları, Storage bucketları ve Row Level Security politikaları uygulamanın temel veri güvenliği katmanını oluşturur.

Bu raporun amacı, React geliştirme sürecinde izlenen yolu akademik çerçevede açıklamak ve projede kullanılan React kavramlarını temel-orta seviye bir bilgi seti halinde özetlemektir.

2. Kullanılan Teknolojiler

Projede kullanılan ana teknolojiler şunlardır:

Teknoloji	Projedeki görevi
React	Kullanıcı arayüzünü component temelli oluşturmak
TypeScript	Tip güvenliği sağlamak ve hata riskini azaltmak
Vite	Geliştirme sunucusu ve production build aracı
React Router	Sayfa yönlendirme ve route koruması
Supabase JS Client	Frontend ile Supabase arasında veri iletişimi
Supabase Auth	Kullanıcı girişi ve oturum yönetimi
Supabase PostgreSQL	Çalışan, izin, maaş, cihaz ve doküman verilerini saklamak
Supabase Storage	Dosya/doküman saklama
ESLint	Kod kalitesi ve standart kontrolü

`package.json` içinde ana scriptler şu şekildedir:

```
npm run dev
npm run build
npm run lint
npm run preview
```

Bu komutlar sırasıyla geliştirme sunucusunu çalıştırmak, production build almak, kod kalitesini kontrol etmek ve build çıktısını önizlemek için kullanılır.

3. React Sürecinde İzlenen Yol

React geliştirme süreci aşamalı ilerlemiştir. Önce temel proje iskeleti kurulmuş, ardından sayfa yapıları, layout, auth sistemi, veri servisleri ve rol bazlı erişim kontrolleri eklenmiştir.

İzlenen genel yol:

1. Vite + React + TypeScript proje temeli oluşturuldu.
2. `src/` klasörü altında uygulama mimarisi ayrıştırıldı.
3. Router yapısı kuruldu.
4. Ortak layout, sidebar ve header componentleri geliştirildi.
5. Tekrarlanabilir UI componentleri oluşturuldu.
6. Supabase client yapılandırıldı.
7. Auth context ve protected route sistemi kuruldu.
8. Feature service dosyaları ile veri erişimi sayfalardan ayrıldı.
9. Çalışan, izin, maaş, cihaz ve doküman sayfaları geliştirildi.
10. Rol bazlı menü ve route erişimi uygulandı.
11. Doküman yükleme/indirme/silme akışları Supabase Storage ile entegre edildi.
12. Canlı Supabase doğrulamaları yapıldı.
13. Login sayfası sadeleştirildi.
14. Uygulama markası HRCore olarak güncellendi.

Bu yaklaşım sayesinde uygulama sadece çalışan bir demo değil, modüler ve sürdürülebilir bir frontend yapısına sahip hale getirilmiştir.

4. Proje Klasör Yapısı

Projedeki frontend organizasyonu şu şekildedir:

```
src/
  app/
    App.tsx
    router.tsx

  components/
    layout/
      AppShell.tsx
      Header.tsx
      Sidebar.tsx
    ui/
      Button.tsx
      Field.tsx
      Modal.tsx
      Panel.tsx
      State.tsx
      ...

  features/
    auth/
      AuthProvider.tsx
      ProtectedRoute.tsx
      authContext.ts
      profileService.ts
      useAuth.ts
    employees/
      employeeService.ts
    leave/
      leaveRequestService.ts
    salary/
      salaryService.ts
    devices/
      deviceService.ts
    documents/
      documentService.ts

  layouts/
    AppLayout.tsx

  lib/
    supabaseClient.ts
    exportCsv.ts

  pages/
    DashboardPage.tsx
    LoginPage.tsx
    EmployeesPage.tsx
    EmployeeDetailPage.tsx
    LeaveRequestsPage.tsx
    SalaryCalculationPage.tsx
    DevicesPage.tsx
    DocumentsPage.tsx
    ProfilePage.tsx
```

```
styles/  
global.css
```

Bu yapı üç temel prensibe dayanır:

- Sayfalar route seviyesinde tutulur.
- Ortak UI parçaları `components/` altında toplanır.
- Veri erişimi ve iş mantığı `features/` altında servis dosyalarına ayrılır.

Bu ayırım, uygulamanın büyüdükçe okunabilir kalmasını sağlar.

5. React Terminolojisi ve Bu Projedeki Karşılıkları

5.1 Component

Component, React uygulamasının tekrar kullanılabilir arayüz parçasıdır. Her component kendi görünümünü ve gerekirse kendi davranışını kapsar.

Bu projede örnek componentler:

- `Button`
- `Field`
- `Panel`
- `Badge`
- `Modal`
- `Sidebar`
- `Header`
- `PageHeader`

Örnek kullanım:

```
<Button variant="primary" icon="upload">  
  Doküman yükle  
</Button>
```

Bu kullanımda `Button` componenti hem görünüm hem davranış açısından standartlaştırılmıştır.

5.2 Props

Props, bir componente dışarıdan gönderilen verilerdir. Componentlerin esnek ve tekrar kullanılabilir olmasını sağlar.

Örneğin `Button` componenti şu propsları alır:

```
variant
size
block
icon
loading
children
```

Bu sayede aynı Button componenti farklı yerlerde farklı biçimlerde kullanılabilir.

5.3 State

State, component içinde değişebilen veridir. Kullanıcı etkileşimi, yükleme durumu, hata mesajı veya form alanları state ile yönetilir.

Örneğin login sayfasında:

```
const [email, setEmail] = useState('')
const [password, setPassword] = useState('')
const [isSubmitting, setIsSubmitting] = useState(false)
```

Bu state değerleri form girişlerini ve submit durumunu yönetir.

5.4 Hook

Hook, React fonksiyon componentlerinde state, lifecycle ve context gibi özellikleri kullanmayı sağlayan fonksiyonlardır.

Bu projede kullanılan temel hooklar:

- `useState`
- `useEffect`
- `useMemo`
- `useCallback`
- `useLocation`
- `useNavigate`
- `useAuth`

`useAuth` bu projeye özel yazılmış custom hooktur.

5.5 useState

`useState`, component içi değişken durumu yönetir. Form alanları, filtreler, loading ve error durumları için kullanılır.

Örneğin dokümanlar sayfasında:

```
const [documents, setDocuments] = useState<Document[]>([])
const [loading, setLoading] = useState(true)
const [error, setError] = useState<string | null>(null)
```

Bu yapı sayfanın veri yükleme sürecini kontrol eder.

5.6 useEffect

`useEffect`, component render olduktan sonra çalışması gereken yan etkileri yönetir. API çağrısı yapmak, session kontrol etmek veya event subscription kurmak için kullanılır.

Bu projede örnek:

```
useEffect(() => {
  void Promise.resolve().then(load)
}, [load])
```

Bu kullanım, sayfa açıldığında ilgili verilerin Supabase'den çekilmesini sağlar.

5.7 useCallback

`useCallback`, fonksiyon referansını gereksiz yere yeniden oluşturmamak için kullanılır. Özellikle `useEffect` dependency listelerinde daha kontrollü davranış sağlar.

Bu projede veri yükleme fonksiyonları genellikle `useCallback` ile sarılmıştır:

```
const load = useCallback(async () => {
  ...
}, [isEmployee, profile, user])
```

5.8 useMemo

`useMemo`, hesaplanan değerleri cachelemek için kullanılır. Bu projede özellikle auth context value oluşturulurken kullanılmıştır:

```
const value = useMemo<AuthContextValue>(() => ({
  ...
}), [errorMessage, isLoading, session, profile])
```

Bu sayede context tüketen componentler gereksiz renderlardan kısmen korunur.

5.9 Context API

Context API, prop drilling yapmadan global benzeri verileri component ağacına dağıtmayı sağlar.

Bu projede auth bilgisi için kullanılmıştır:

```
AuthProvider
-> AuthContext
-> useAuth
```

Bu yapı sayesinde herhangi bir sayfa şu verilere erişebilir:

- Kullanıcı giriş yapmış mı?
- Aktif session var mı?
- Kullanıcı profili nedir?
- Kullanıcının rolü nedir?
- Sign in / sign out fonksiyonları

5.10 Custom Hook

Custom hook, tekrar kullanılabilir React mantığını paketleyen özel hooktur.

Bu projede:

```
useAuth()
```

Auth context'e erişimi kolaylaştırmak için kullanılmıştır.

5.11 Router

Router, uygulamada URL'e göre hangi sayfanın gösterileceğini belirler.

Bu projede `react-router-dom` kullanılmıştır:

```
createBrowserRouter([...])
```

Ana route yapısı:

```
/login
/dashboard
/profile
/employees
/employees/:id
/leave-requests
/salary-calculation
/devices
/documents
```

5.12 Protected Route

Protected Route, sadece belirli kullanıcıların erişebileceği sayfaları korumak için kullanılır.

Bu projede iki katman vardır:

1. Giriş yapmış kullanıcı kontrolü.

2. Rol bazlı erişim kontrolü.

Örnek:

```
<ProtectedRoute allowedRoles={['admin_hr']} />
```

Bu yapı sayesinde maaş hesaplama ekranı sadece Admin/HR rolüne açılır.

5.13 Layout

Layout, sayfaların ortak kabuğudur. Sidebar, header ve ana içerik alanı layout içinde yönetilir.

Bu projede:

- AppLayout
- AppShell
- Sidebar
- Header

kullanılmıştır.

Bu sayede tüm authenticated sayfalar aynı navigasyon yapısını paylaşır.

5.14 Service Layer

Service layer, API/veri erişim kodlarını UI componentlerinden ayırır.

Bu projede örnek servisler:

- employeeService.ts
- leaveRequestService.ts
- salaryService.ts
- deviceService.ts
- documentService.ts
- profileService.ts

Örneğin bir sayfa doğrudan Supabase sorgusu yazmak yerine şunu çağırır:

```
const docs = await getDocuments()
```

Bu yöntem kod tekrarını azaltır ve bakım kolaylığı sağlar.

5.15 TypeScript Type

TypeScript type tanımları, verinin şeklini belirler. Bu projede Supabase tablolarına karşılık gelen tipler servis dosyalarında tanımlanmıştır.

Örnek:


```
export type Document = {
  id: string
  employee_id: string | null
  document_type: 'cv' | 'employee_document' | 'device_invoice' | 'assignment_document' | 'other'
  file_name: string
  storage_bucket: string
  storage_path: string
}
```

Bu sayede yanlış alan kullanımı veya hatalı veri tipi build aşamasında yakalanabilir.

5.16 Conditional Rendering

Conditional rendering, duruma göre farklı arayüz göstermek demektir.

Bu projede sık kullanılan durumlar:

- Loading ise skeleton göster.
- Error varsa hata göster.
- Liste boşsa EmptyState göster.
- Rol employee ise kendi verisini göster.
- Rol admin ise tüm veriyi göster.

Örnek akış:

```
loading ? (
  <SkeletonRow />
) : error ? (
  <ErrorState />
) : filtered.length === 0 ? (
  <EmptyState />
) : (
  <table>...</table>
)
```

5.17 Controlled Form

Controlled form, input değerlerinin React state üzerinden yönetilmesidir.

Login formu örneği:

```
<Input
  value={email}
  onChange={e => setEmail(e.target.value)}
/>
```

Bu yaklaşım form verisini kontrol altında tutar.

5.18 Event Handling

Event handling, kullanıcı aksiyonlarına tepki vermektir.

Bu projedeki örnek eventler:

- `onClick`
- `onChange`
- `onSubmit`
- `onDrop`
- `onDragOver`

Örneğin doküman yükleme ekranında drag/drop davranışı kullanılmıştır.

5.19 Async/Await

Supabase işlemleri asenkron çalışır. Bu nedenle `async/await` kullanılmıştır.

Örnek:

```
const doc = await uploadEmployeeDocument(...)
```

Bu kullanım, veri işlemlerinin okunabilir olmasını sağlar.

5.20 Error, Loading ve Empty State

Profesyonel uygulamalarda sadece başarılı veri durumu değil, yüklenme, hata ve boş liste durumları da tasarlanmalıdır.

Bu projede:

- `SkeletonRow`
- `ErrorState`
- `EmptyState`

componentleri kullanılmıştır.

Bu yaklaşım kullanıcı deneyimini güçlendirir.

6. Auth ve Rol Bazlı Yetkilendirme

Projede kimlik doğrulama Supabase Auth ile yapılır. Kullanıcı giriş yaptığında Supabase bir session üretir. React tarafında bu session `AuthProvider` içinde tutulur.

Kimlik zinciri:

```
Supabase Auth user
-> auth.users.id
-> profiles.id
-> profiles.role
-> employees.profile_id
```

Bu zincir sayesinde uygulama şu sorulara cevap verir:

- Kullanıcı giriş yaptı mı?
- Kullanıcının profili var mı?
- Kullanıcının rolü nedir?
- Kullanıcı bir çalışan kaydıyla eşleşiyor mu?

Roller:

Rol	Yetki kapsamı
admin_hr	Tüm yönetim ekranlarına erişebilir
manager	Kendi takımını ve takım izinlerini görebilir
employee	Kendi profilini, izinlerini, cihazlarını ve dokümanlarını görebilir

Frontend tarafında `ProtectedRoute` kullanılır. Ancak gerçek veri güvenliği Supabase RLS politikaları ile sağlanmalıdır. Frontend route guard kullanıcı deneyimini düzenler, backend güvenliği ise RLS ile garanti altına alınır.

7. Sayfa Bazlı React Akışları

7.1 LoginPage

Login sayfası kullanıcıdan e-posta ve şifre alır. `signIn` fonksiyonu çağrılır ve başarılı girişten sonra kullanıcı dashboard'a yönlendirilir.

Bu sayfada kullanılan React kavramları:

- `useState`
- Controlled inputs
- `onSubmit`
- Conditional error rendering
- `useNavigate`
- `Navigate`

Son aşamada login sayfası sadeleştirilmiş, tanıtım kartları ve gereksiz linkler kaldırılmıştır.

7.2 DashboardPage

Dashboard, role göre farklı özetler gösterir. Admin için genel metrikler, employee için kişisel metrikler önemlidir.

Kullanılan kavramlar:

- Role-based rendering
- Async data fetching
- StatCard componentleri
- Error/loading state

7.3 EmployeesPage

Çalışan listeleme ve çalışan oluşturma işlemlerini kapsar. Admin/HR tüm çalışanları görebilir. Manager kendi takımıyla sınırlıdır.

Kullanılan kavramlar:

- Table rendering
- Modal/form kullanımı
- Service layer ile veri çekme
- Role-based filtering

7.4 EmployeeDetailPage

Tek bir çalışanın detaylarını gösterir. Çalışanın profil bilgisi, cihazları, izinleri ve dokümanları burada görülebilir.

Kullanılan kavramlar:

- URL parametreleri
- Detail view
- Tabs
- Conditional action buttons
- Related data fetching

7.5 LeaveRequestsPage

İzin taleplerini listeler, yeni izin talebi oluşturur ve manager için onay/red akışı sağlar.

Kullanılan kavramlar:

- Form state
- Role-based action
- Filtered data
- Status badges

7.6 SalaryCalculationPage

Maaş hesaplama ve kayıt oluşturma ekranıdır. Sadece Admin/HR rolüne açıktır.

Kullanılan kavramlar:

- Protected route
- Derived calculation
- Form validation
- Service call

7.7 DevicesPage

Cihaz oluşturma, cihaz atama ve iade süreçlerini kapsar.

Kullanılan kavramlar:

- CRUD benzeri işlemler
- Assignment flow
- Role-based visible actions
- Table filtering

7.8 DocumentsPage

Doküman yükleme, indirme ve silme işlemlerini kapsar. Supabase Storage ile entegredir.

Doküman akışı:

```
File seçilir
-> Supabase Storage'a upload edilir
-> public.documents tablosuna metadata kaydı açılır
-> İndirme için signed URL üretilir
-> Silmede metadata ve storage objesi kaldırılır
```

Bu modülde kullanılan kavramlar:

- File input
- Drag/drop upload
- Async upload
- Signed URL
- Delete action
- Optimistic olmayan güvenli UI güncellemesi

Silme akışında önemli bir düzeltme yapılmıştır: Supabase delete çağrısı gerçekten satır sildiyse UI listeden kaldırır. Eğer RLS nedeniyle satır silinmediyse kullanıcıya hata gösterilir.

8. Supabase ile React Entegrasyonu

Supabase client şu dosyada yapılandırılır:

```
src/lib/supabaseClient.ts
```

Temel mantık:

```
const supabase = createClient(url, anonKey)
```

Frontend sadece public anon key kullanır. Service role key frontend'e konmaz. Bu güvenlik açısından önemlidir.

Supabase ile yapılan işlem türleri:

- `select`
- `insert`
- `update`
- `delete`
- `storage.upload`
- `storage.remove`
- `createSignedUrl`
- `auth.signInWithPassword`
- `auth.signOut`
- `auth.getSession`
- `auth.onAuthStateChange`

9. Canlı Doğrulama Süreci

Proje sadece local build ile bırakılmamış, canlı Supabase üzerinde de doğrulanmıştır.

Yapılan kontroller:

1. `.env.local` içindeki Supabase URL ve anon key kontrol edildi.
2. Anon erişimde tabloların açık olup olmadığı test edildi.
3. Admin/HR, Manager ve Employee hesaplarıyla login testi yapıldı.
4. `auth.users.id -> profiles.id -> employees.profile_id` eşleşmesi doğrulandı.
5. Employee profil eşleşmesindeki eksik kayıt düzeltildi.
6. Storage upload/download smoke test yapıldı.
7. RLS boundary testi yapıldı.
8. Employee başka çalışanı okuyamıyor mu kontrol edildi.
9. Manager takım dışı çalışana erişemiyor mu kontrol edildi.
10. Doküman metadata ve storage path tutarlılığı incelendi.

11. Doküman silme akışında RLS DELETE policy ihtiyacı tespit edildi.

Önemli doğrulama sonucu:

```
employee_read_self=ALLOW
employee_read_other=DENY
manager_read_team_member=ALLOW
manager_read_non_team_member=DENY
```

Bu sonuçlar RLS politikalarının temel sınırları doğru uyguladığını gösterir.

10. Projede Yapılan Önemli İyileştirmeler

10.1 Employee Profile Eşlemesi

Başta employee test hesabında `profiles` kaydı eksikti. Bu nedenle sistem e-posta fallback'i ile çalışanı buluyordu. Daha doğru zincir için profile kaydı oluşturuldu ve employee kaydı `profile_id` ile bağlandı.

Doğru zincir:

```
auth.users.id = profiles.id = employees.profile_id
```

10.2 Doküman Silme

Dokümanların sadece yüklenip indirilen pasif kayıtlar olması yeterli görülmedi. Kullanıcının kendi dokümanlarını silebilmesi istendi.

Frontend tarafında:

- Silme butonu eklendi.
- Onay penceresi eklendi.
- Gerçek delete doğrulaması eklendi.

Backend tarafında employee DELETE policy gereksinimi tespit edildi.

10.3 Login Sayfası Sadeleştirme

Login sayfasındaki tanıtım kartları, metrikler, SSO linki, footer linkleri, "beni hatırla" ve "şifremi unuttum" alanları kaldırıldı.

Son durumda login sayfası sade bir giriş formuna indirildi:

- HRCore marka alanı
- E-posta
- Şifre
- Giriş yap butonu

10.4 Marka Değişimi

Uygulama adı **Northwind İK** yerine **HRCore** yapıldı. Sembol **N** yerine **HRC** monogramı olarak güncellendi.

Güncellenen yerler:

- Login sayfası
- Sidebar
- Browser title
- README
- CSS yorumları

11. React'te Bilinmesi Gereken Temel-Orta Seviye Kavramlar

Bu projeyi savunmak veya geliştirmek için şu kavramlara hakim olmak önemlidir:

Kavram	Kısa açıklama	Projedeki örnek
Component	Arayüz parçası	Button, Panel, Sidebar
Props	Componente dış veri geçme	Button variant, icon
State	Component içi değişken veri	Form alanları, loading
Hook	React özelliklerini kullanma yolu	useState, useEffect
Custom Hook	Özel tekrar kullanılabilir hook	useAuth
Context	Global veri paylaşımı	AuthContext
Router	URL'e göre sayfa seçimi	router.tsx
Protected Route	Yetkili sayfa erişimi	ProtectedRoute
Layout	Ortak sayfa iskeleti	AppLayout
Service Layer	Veri erişim katmanı	employeeService
Conditional Rendering	Duruma göre UI gösterme	Loading/Error/Empty
Controlled Input	Input değerini state ile yönetme	Login form
Async/Await	Asenkron veri işlemleri	Supabase çağrıları
Type	Veri şeklini tanımlama	Document, Profile
RLS	Backend veri güvenliği	Supabase policies

12. Bu Projede Kazanılan Yetkinlikler

Bu projede geliştirici olarak aşağıdaki yetkinlikler kazanılmıştır:

- React component mimarisi kurma
- TypeScript ile tip güvenli frontend geliştirme
- React Router ile sayfa yönlendirme
- Protected route ve role-based access control uygulama
- Context API ile global auth state yönetme
- Supabase Auth entegrasyonu yapma
- Supabase PostgreSQL tablolarıyla frontend veri akışı kurma
- Supabase Storage ile dosya yükleme/indirme/silme akışı geliştirme
- RLS mantığını frontend davranışıyla birlikte değerlendirme
- Loading, error ve empty state tasarlama
- Form state ve validation yönetme
- Modüler klasör yapısı oluşturma
- Production build ve lint kontrolleri yapma
- Canlı sistem doğrulama scriptleriyle backend davranışını test etme

13. Akademik Değerlendirme

HRCore projesi akademik açıdan sadece bir arayüz çalışması değildir. Proje, modern frontend geliştirme prensiplerini ve backend güvenlik yaklaşımını birlikte ele alır.

Öne çıkan teknik değerler:

- Component tabanlı mimari
- Modüler servis katmanı
- Rol bazlı erişim kontrolü
- Canlı backend entegrasyonu
- Dosya depolama entegrasyonu
- RLS ile veri güvenliği
- TypeScript ile sürdürülebilir kod
- Build/lint doğrulaması

Bu yönleriyle proje, temel bir CRUD uygulamasının ötesine geçerek gerçek hayattaki bir İK yönetim panelinin MVP seviyesindeki ihtiyaçlarını karşılayacak yapıya yaklaşmıştır.

14. Kalan Teknik Notlar

Teslim öncesinde dikkat edilmesi gereken bazı noktalar:

1. Employee doküman silme için Supabase DELETE policy eklenmelidir.

2. Eski bir doküman metadata kaydının storage objesi eksiktir; dosya yeniden yüklenmeli veya UI bu durumu açık göstermelidir.
3. `docs/README.md` içindeki silinmiş doküman linkleri temizlenmelidir.
4. Final teslimden önce `npm run lint` ve `npm run build` tekrar çalıştırılmalıdır.
5. Git çalışma ağacı temizlenmeli ve bilinçli commit alınmalıdır.

15. Sonuç

React sürecinde HRCore projesi component temelli, route kontrollü, auth destekli ve Supabase entegre bir web uygulaması haline getirilmiştir. Projede React'in temel kavramları olan component, props, state, hooks, context, router ve conditional rendering aktif biçimde kullanılmıştır. Orta seviye tarafta ise protected route, service layer, role-based rendering, async data fetching ve Supabase Storage entegrasyonu uygulanmıştır.

Bu proje üzerinden React bilgisi sadece teorik düzeyde değil, gerçek bir uygulama mimarisi içinde pratik olarak öğrenilmiş ve uygulanmıştır.